

QpiAI™ - Joint AI Training and Certification Program

Variational Autoencoders

Sourjya Sarkar

Generative Models

Give training data, generate new samples from same distribution



Training data $\sim p_{\text{data}}(X)$



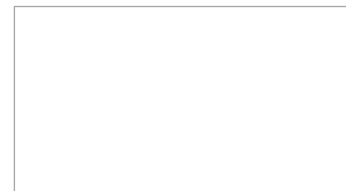
Generated samples $\sim p_{\text{model}}(X)$

Want to learn $p_{\text{model}}(X)$ similar to $p_{\text{data}}(X)$

Addresses density estimation, a core problem in unsupervised learning

Two Types:

- Explicit density estimation: explicitly define and solve for $p_{\text{model}}(X)$
- Implicit density estimation: learn a model that can sample from $p_{\text{model}}(X)$ without explicitly defining it



Why Generative Models?

Discriminative models

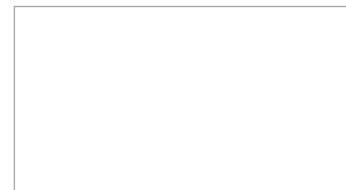
- Given an image $\mathbf{X}$, predict a label $\mathbf{Y}$
- Estimates $\mathbf{P}(\mathbf{Y}|\mathbf{X})$

Discriminative models have several key limitations

- Can't model $\mathbf{P}(\mathbf{X})$, i.e. the probability of seeing a certain image
- This, can't sample from $\mathbf{P}(\mathbf{X})$, i.e. **can't generate new images**

Generative models (in general) cope with all of above

- Can model $\mathbf{P}(\mathbf{X})$
- Can generate new images

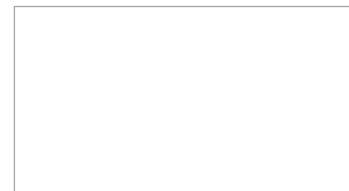


Why Generative Models?

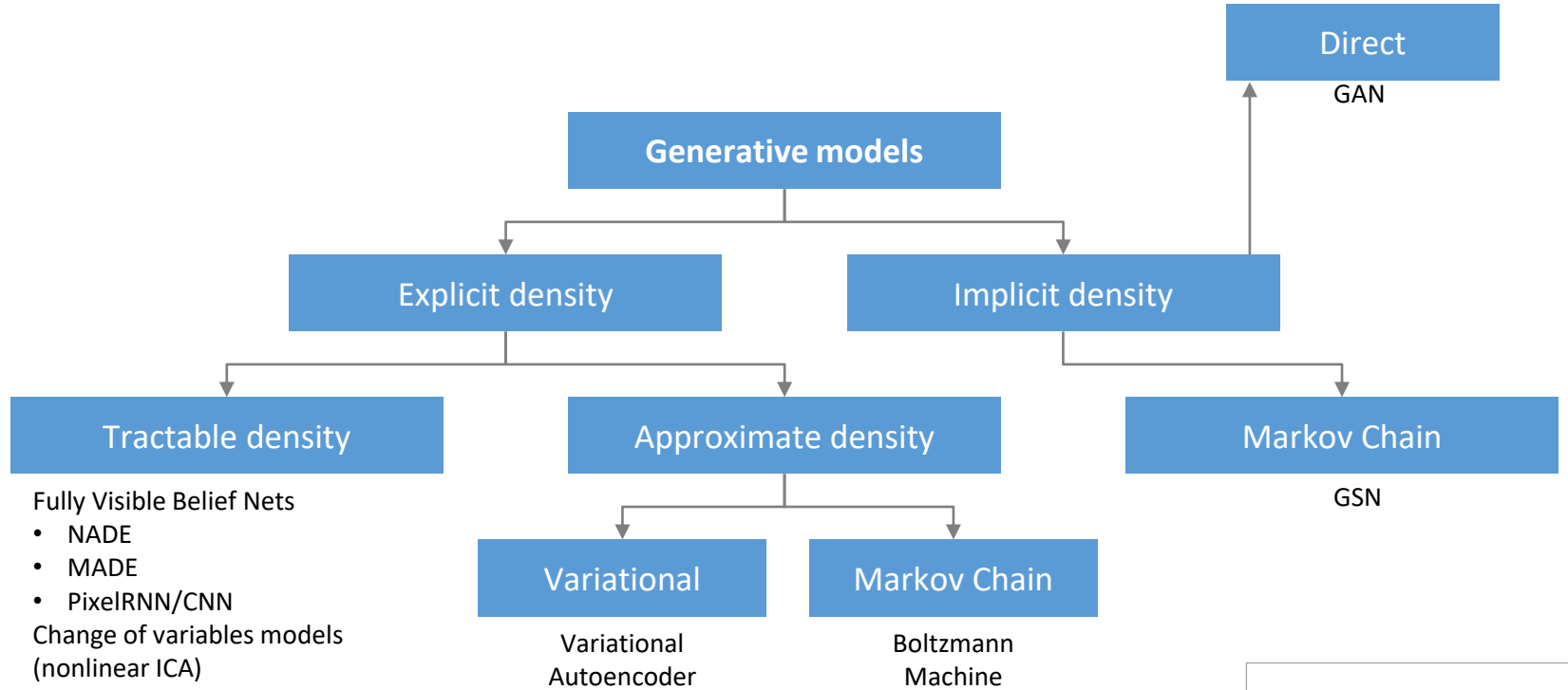
Realistic samples for artwork, super-resolution, colorization, etc.



- Generative models of time-series data can be used for simulation and planning (reinforcement learning applications)
- Training generative models can also enable inference of latent representations that can be useful as general features

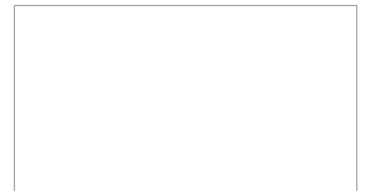


Taxonomy of Generative Models



Variational Autoencoders

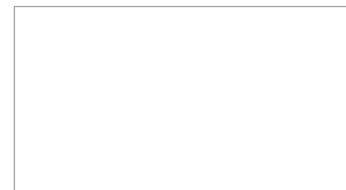
- **Mathematical Preliminaries**
- Autoencoders and Issues
- Derivations
- Reparameterization
- Examples and Summary



Posterior Inference

Posterior via Bayes' Rule:

$$p(z|x) = \frac{p_{\theta}(x|z)p(z)}{\int p_{\theta}(x|z)p(z)dz}$$



Expectation

- Give a random variable X with probability density $p(x; \theta)$ (with parameters θ), the expectation is give by

$$\mathbb{E}[X] = \int xp(x; \theta)dx$$

- Expectation of a function $f(x; \phi)$ with respect to probability distribution $p(x; \theta)$ is given by

$$\mathbb{E}_{p(x; \theta)}[f(x; \phi)] = \int p(x; \theta)f(x; \phi)dx$$

- Monte Carlo approximation of Expectation.

$$\mathbb{E}_{p(x; \theta)}[f(x; \phi)] = \frac{1}{N} \sum_{x^i \sim p(x; \theta)} f(x^i; \phi)$$

- Linearity of Expectation

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$$

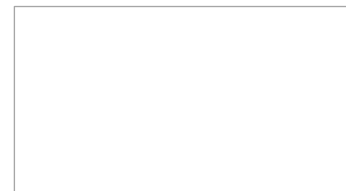
- If X and Y are independent random variables

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$$

Kullback-Leibler Divergence

$$\begin{aligned} D_{\text{KL}}(q \parallel p) &= - \int q(z) \log \left(\frac{p(z)}{q(z)} \right) dz \\ &= \mathbb{E}_q \left[-\log \left(\frac{p}{q} \right) \right] \end{aligned}$$

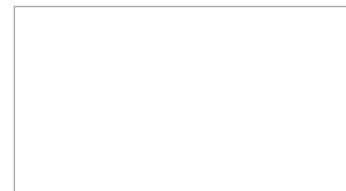
The average information gained from moving from q to p



Variational Inference

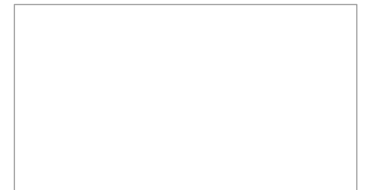
Approximate intractable posterior $p(z|x)$ with a manageable distribution $q(z)$

Minimize the KL divergence: $D_{KL}(q(z) \parallel p(z|x))$

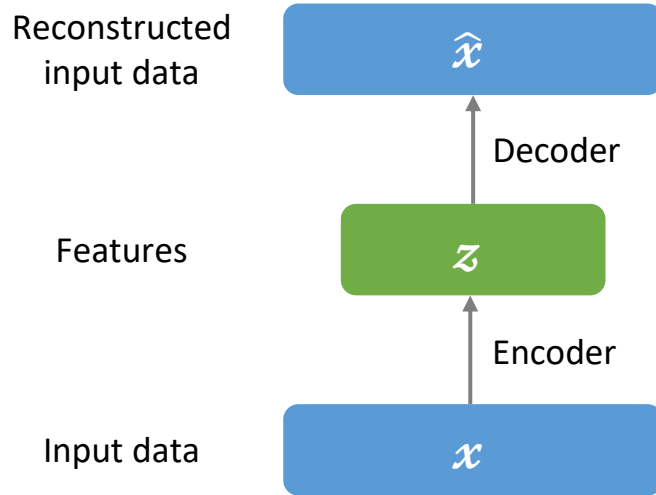


Variational Autoencoders

- Mathematical Preliminaries
- **Autoencoders and Issues**
- Derivations
- Reparameterization
- Examples and Summary

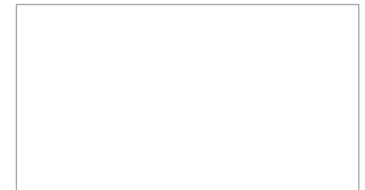


Background: Autoencoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data. Can we generate new images from an autoencoder?



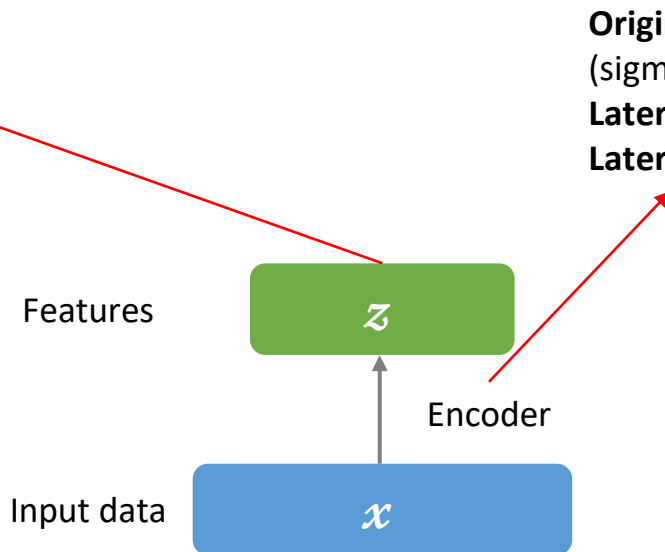
Background: Autoencoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

$\mathbf{z}$ usually smaller than $\mathbf{x}$
(dimensionality reduction)

Q: Why dimensionality reduction?

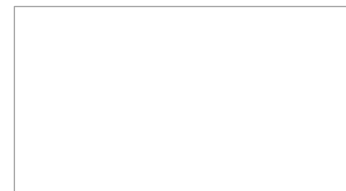
A: What features to capture meaningful factor of variation in data



Originally: Linear + nonlinearity (sigmoid)

Later: Deep, fully-connected

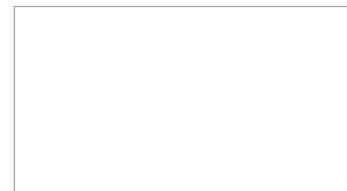
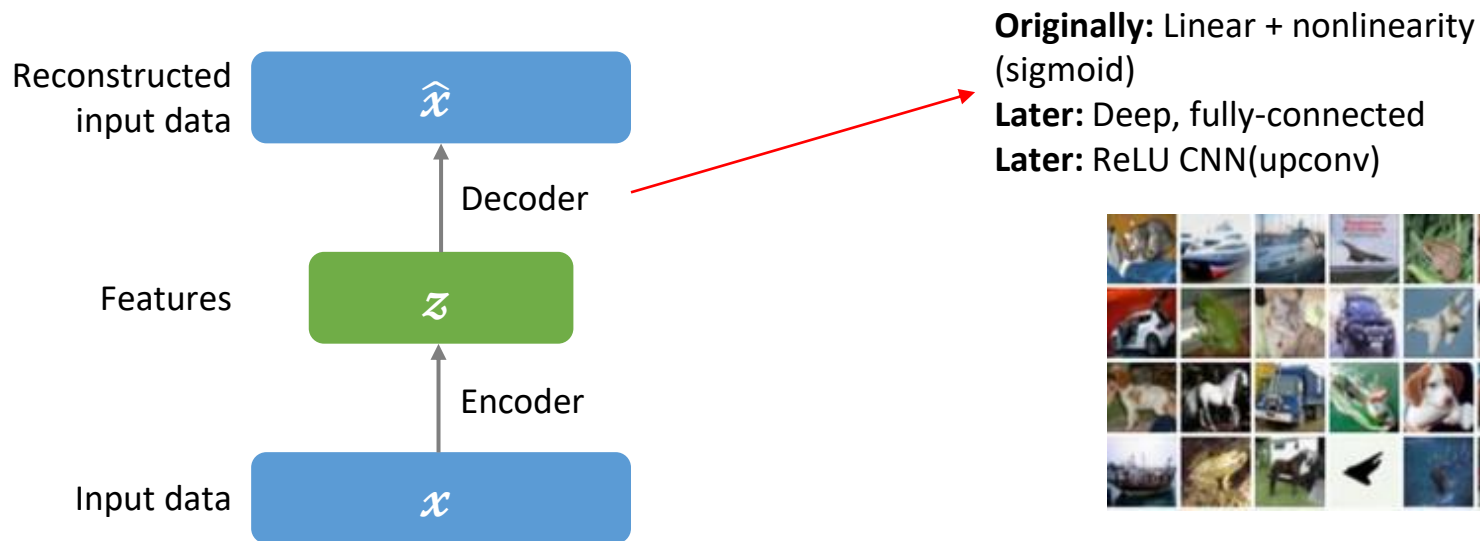
Later: ReLU CNN



Background: Autoencoders

How to learn this feature representation?

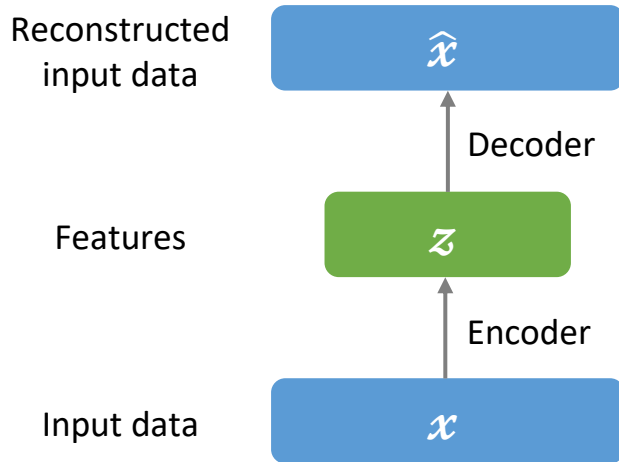
Train such that features can be used to reconstruct original data “Autoencoding” – encoding itself



Background: Autoencoders

How to learn this feature representation?

Train such that features can be used to reconstruct original data
“Autoencoding” – encoding itself

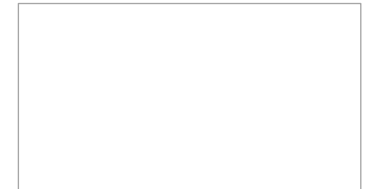


Reconstructed data



Encoder: 4-layer conv
Decoder: 4-layer upconv

Input data

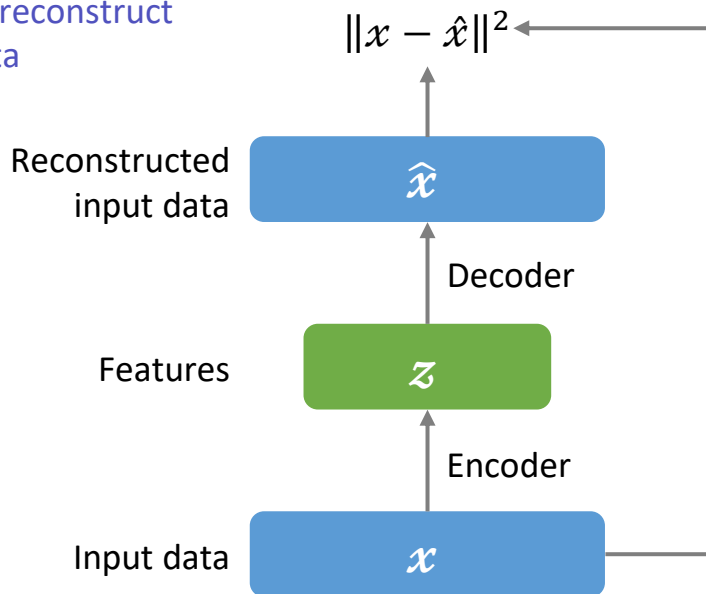


Background: Autoencoders

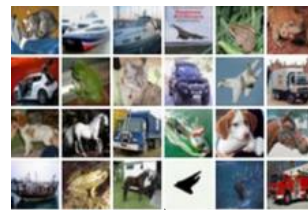
Train such that features can be used to reconstruct original data

L2 Loss function:

Don't use labels!



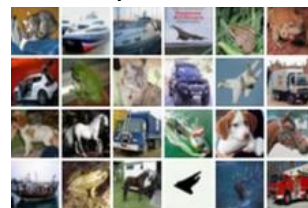
Reconstructed data



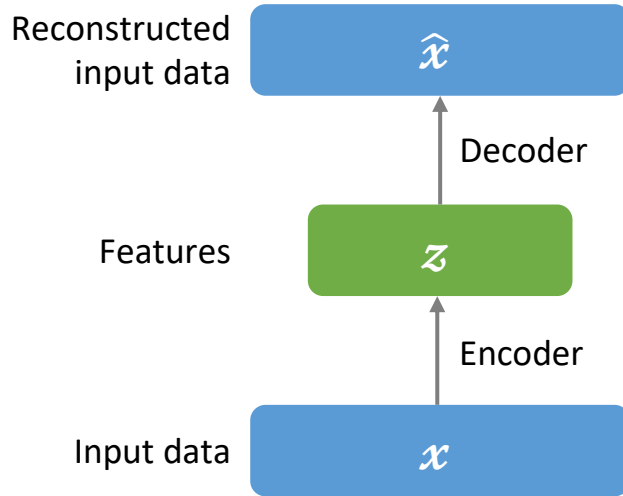
Encoder: 4-layer conv

Decoder: 4-layer upconv

Input data

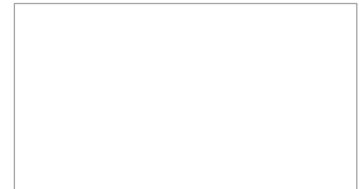


Background: Autoencoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data. Can we generate new images from an autoencoder?



Variational Autoencoders

Probabilistic spin on autoencoders – will let us sample from the model to generate data!

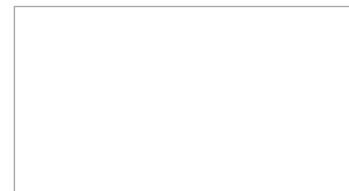
Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from underlying unobserved (latent) representation $\mathbf{Z}$

Sample from true
conditional
 $p_{\theta^*}(x|z^{(i)})$



Intuition (remember from autoencoders!):
 $\mathbf{X}$ is an image, $\mathbf{Z}$ is latent factors used to
generate $\mathbf{X}$: attributes, orientation, etc.

Sample from true
prior
 $p_{\theta^*}(z)$



Variational Autoencoders

Sample from true conditional
 $p_{\theta^*}(x|z^{(i)})$



Sample from true prior
 $p_{\theta^*}(z)$



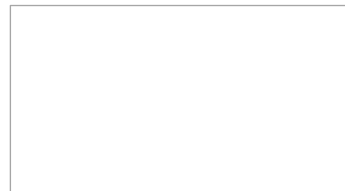
Decoder
network

We want to estimate the true parameters θ^* of this generative model.

How should we represent this model?

Choose prior $p(z)$ to be simple, e.g. Gaussian.

Conditional $p(x|z)$ is complex (generates image) => represent with neural network



Variational Autoencoders

Sample from true conditional

$$p_{\theta^*}(x|z^{(i)})$$



Decoder
network

Sample from true prior

$$p_{\theta^*}(z)$$



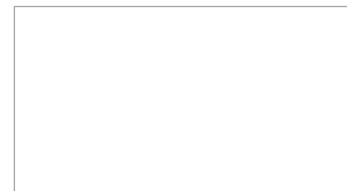
We want to estimate the true parameters θ^* of this generative model.

How to train the model?

Learn model parameters to maximize likelihood of training data

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

Now with latent z



Variational Autoencoders

Sample from true conditional

$$p_{\theta^*}(x|z^{(i)})$$



Decoder
network

Sample from true prior

$$p_{\theta^*}(z)$$



We want to estimate the true parameters θ^* of this generative model.

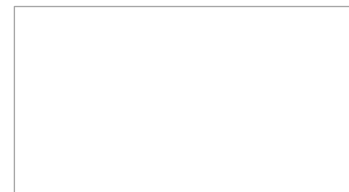
How to train the model?

Learn model parameters to maximize likelihood of training data

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

Q: What is the problem with this?

Intractable!



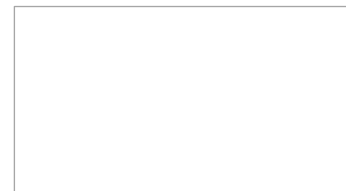
Variational Autoencoders: Intractability

Data likelihood:


$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

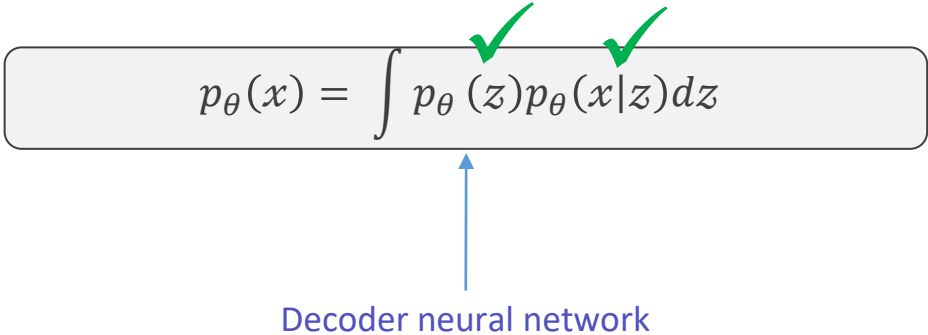


Simple Gaussian prior

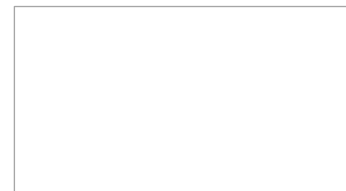


Variational Autoencoders: Intractability

Data likelihood:

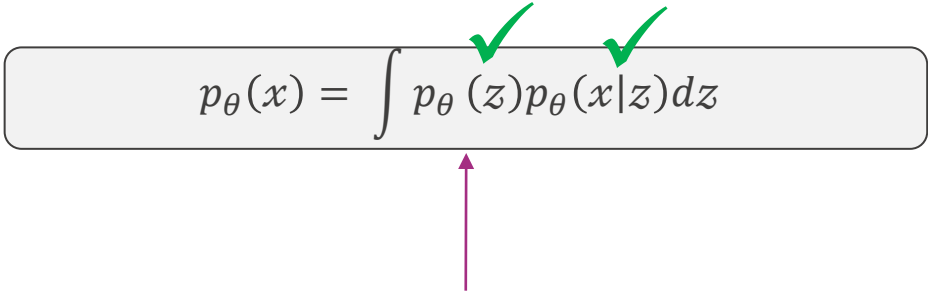
$$p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$$


Decoder neural network

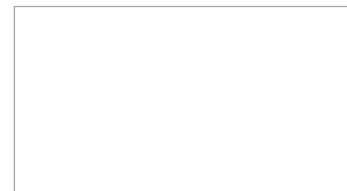


Variational Autoencoders: Intractability

Data likelihood:

$$p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$$


Intractable to compute
 $p(x|z)$ for every z !



Variational Autoencoders: Intractability

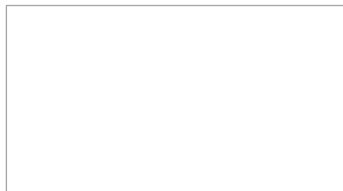
Data likelihood:

$$p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$$

Posterior density also intractable:

$$p_{\theta}(z|x) = p_{\theta}(x|z)p_{\theta}(z)/p_{\theta}(x)$$

Intractable data likelihood



Variational Autoencoders: Intractability

Data likelihood:

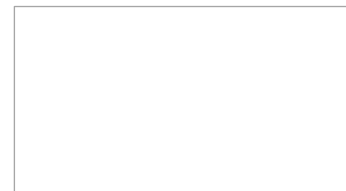
$$p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$$

Posterior density also intractable:

$$p_{\theta}(z|x) = p_{\theta}(x|z)p_{\theta}(z)/p_{\theta}(x)$$

Solution: In addition to decoder network modeling $p_{\theta}(x|z)$, define additional encoder network $q_{\phi}(z|x)$ that approximates $p_{\theta}(z|x)$

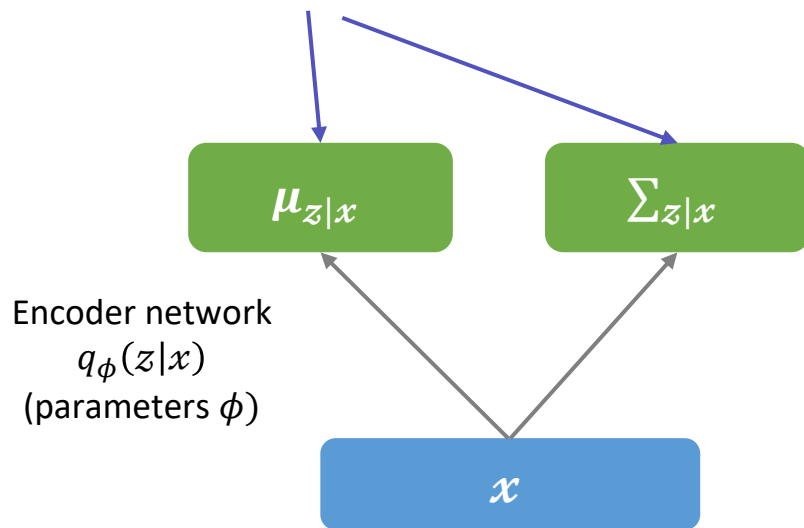
Will see that this allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize



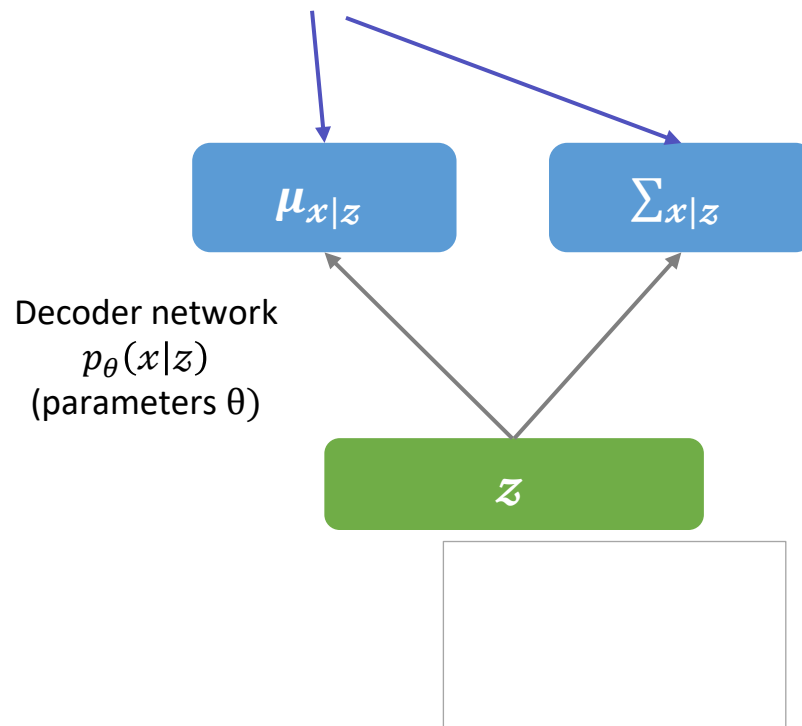
Variational Autoencoders

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic

Mean and (diagonal) covariance of $\mathbf{z} \mid \mathbf{x}$

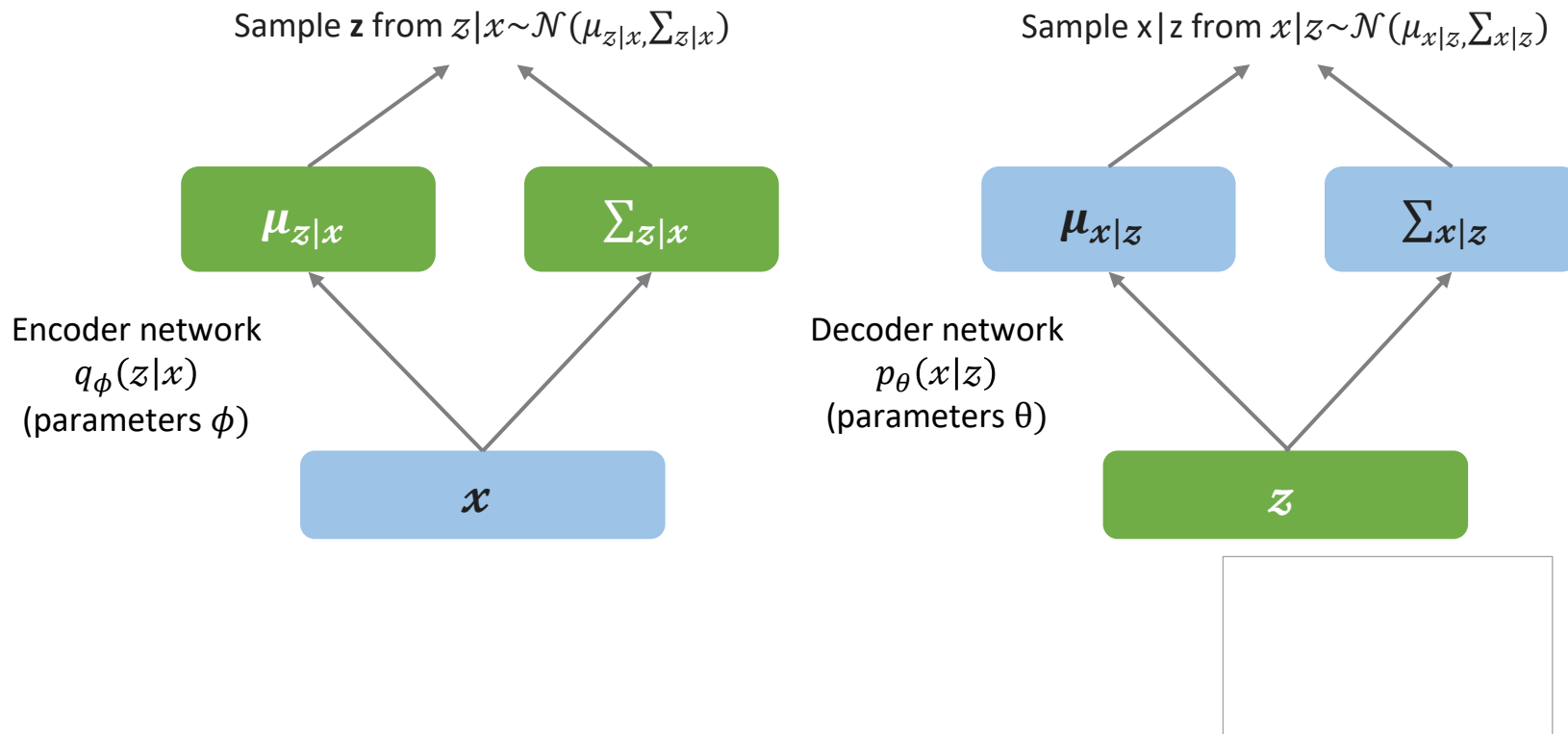


Mean and (diagonal) covariance of $\mathbf{x} \mid \mathbf{z}$

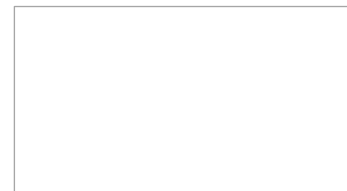
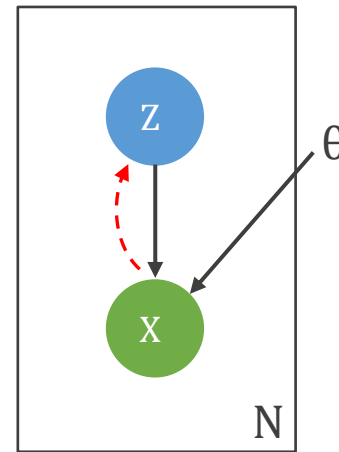
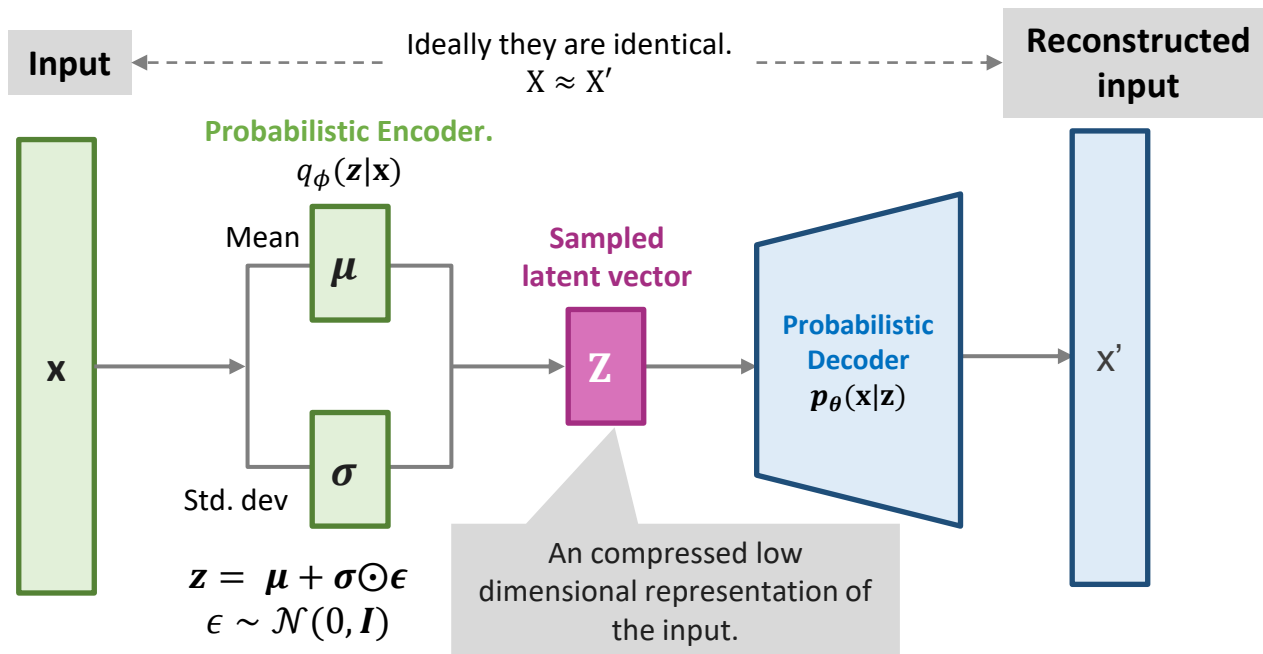


Variational Autoencoders

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic

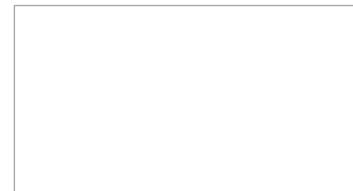


VAE Diagram



Variational Autoencoders

- Mathematical Preliminaries
- Autoencoders and Issues
- **Derivations**
- Reparameterization
- Examples and Summary



VAE Derivations

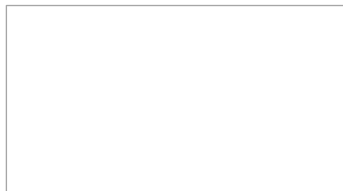
Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\begin{aligned}
 \log p_{\theta}(x^{(i)}) &= \mathbb{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})] && (p_{\theta}(x^{(i)}) \text{ Does not depend on } z) \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right] && (\text{Bayes' Rule}) \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \frac{q_{\phi}(z|x^{(i)})}{q_{\phi}(z|x^{(i)})} \right] && (\text{Multiple constant}) \\
 &= \mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right] && (\text{Logarithms}) \\
 &= \mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - D_{\text{KL}}(q_{\phi}(z|x^{(i)}) \parallel p_{\theta}(z)) + D_{\text{KL}}(q_{\phi}(z|x^{(i)}) \parallel p_{\theta}(z|x^{(i)}))
 \end{aligned}$$

Decoder network gives $p_{\theta}(x|z)$, can compute estimate of this term through sampling. (Make sampling differentiable through reparameterization trick.)

This KL term (between Gaussians for encoder and z prior) has nice closed-form solution!

$p_{\theta}(z|x)$ intractable (saw earlier), can't compute this KL term : (But we know KL divergence always ≥ 0 .)



VAE Derivations

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

↑
We want to maximize the data likelihood

$$\begin{aligned}
 \log p_{\theta}(x^{(i)}) &= \mathbb{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})] && (p_{\theta}(x^{(i)})) \text{ Does not depend on } z \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right] && (\text{Bayes' Rule}) \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \frac{q_{\phi}(z|x^{(i)})}{q_{\phi}(z|x^{(i)})} \right] && (\text{Multiple constant}) \\
 &= \mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right] && (\text{Logarithms}) \\
 &= \underbrace{\mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - D_{\text{KL}}(q_{\phi}(z|x^{(i)}) \parallel p_{\theta}(z))}_{\mathcal{L}(x^{(i)}; \theta, \phi)} + \underbrace{D_{\text{KL}}(q_{\phi}(z|x^{(i)}) \parallel p_{\theta}(z|x^{(i)}))}_{\geq 0}
 \end{aligned}$$

Tractable lower bound which we can take gradient of and optimize! ($p_{\theta}x|z$) differentiable, KL term differentiable)

VAE Derivations

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\begin{aligned}
 \log p_{\theta}(x^{(i)}) &= \mathbb{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})] && (p_{\theta}(x^{(i)}) \text{ Does not depend on } z) \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right] && \text{(Bayes' Rule)} \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \frac{q_{\phi}(z|x^{(i)})}{q_{\phi}(z|x^{(i)})} \right] && \text{(Multiple constant)} \\
 &= \mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right] && \text{(Logarithms)} \\
 &= \underbrace{\mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - D_{KL}(q_{\phi}(z|x^{(i)}) \parallel p_{\theta}(z))}_{\mathcal{L}(x^{(i)}; \theta, \phi)} + \underbrace{D_{KL}(q_{\phi}(z|x^{(i)}) \parallel p_{\theta}(z|x^{(i)}))}_{\geq 0}
 \end{aligned}$$

Reconstruct the input data

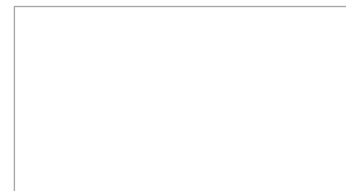
Make approximate posterior distribution close to prior

$\log p_{\theta}(x^{(i)}) \geq \mathcal{L}(x^{(i)}; \theta, \phi)$
Evidence (Variational) lower bound ('ELBO')

$\theta^*, \phi^* = \arg \max_{\theta, \phi} \sum_{i=1}^N \mathcal{L}(x^{(i)}; \theta, \phi)$
Training: Maximize lower bound

Variational Autoencoders

- Mathematical Preliminaries
- Autoencoders and Issues
- Derivations
- **Reparameterization**
- Examples and Summary



Evidence Lower Bound (ELBO)

The ELBO was defined as

$$\mathcal{L}(x; \theta, \phi) = \mathbb{E}_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - \text{KL}[q_{\phi}(z|x) || p_{\theta}(z)]$$

If we rearrange the terms above, we get

$$\begin{aligned}\mathcal{L}(x; \theta, \phi) &= \int q_{\phi}(z|x) \log p_{\theta}(x|z) dz - \int q_{\phi}(z|x) \log \left(\frac{q_{\phi}(z|x)}{p_{\theta}(z)} \right) dz \\&= \int q_{\phi}(z|x) \log p_{\theta}(x|z) dz - \int q_{\phi}(z|x) \log q_{\phi}(z|x) dz + \int q_{\phi}(z|x) \log p_{\theta}(z) dz \\&= \int q_{\phi}(z|x) [\log p_{\theta}(x|z) + \log p_{\theta}(z)] dz - \int q_{\phi}(z|x) \log q_{\phi}(z|x) dz \\&= \int q_{\phi}(z|x) [\log p_{\theta}(x, z)] dz - \int q_{\phi}(z|x) \log q_{\phi}(z|x) dz \\&= \mathbb{E}_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)]\end{aligned}$$

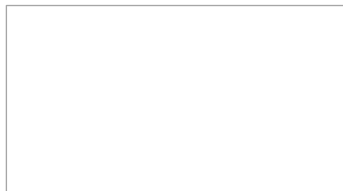
Gradients

- An unbiased gradient estimator of the ELBO w.r.t. the generative model parameters θ is straightforwardly obtained:

$$\begin{aligned}\nabla_{\theta} \mathcal{L}_{\theta, \phi}(x) &= \nabla_{\theta} \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)] \\ &= \mathbb{E}_{q_{\phi}(z|x)} \left[\nabla_{\theta} (\log p_{\theta}(x, z) - \log q_{\phi}(z|x)) \right] \\ &\simeq \nabla_{\theta} (\log p_{\theta}(x, z) - \log q_{\phi}(z|x))\end{aligned}$$

- A gradient estimator of the ELBO w.r.t. the variational parameters ϕ is more difficult to obtain:

$$\begin{aligned}\nabla_{\phi} \mathcal{L}_{\theta, \phi}(x) &= \nabla_{\phi} \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)] \\ &\neq \mathbb{E}_{q_{\phi}(z|x)} \left[\nabla_{\phi} (\log p_{\theta}(x, z) - \log q_{\phi}(z|x)) \right]\end{aligned}$$



Reparameterization Trick

- Construct the following Monte Carlo estimator:

$$\begin{aligned}\epsilon &\sim p(\epsilon) \\ z &= g(\epsilon, \phi, X) \\ \tilde{\mathcal{L}}_{\theta, \phi}(x; \epsilon) &= \log p_{\theta}(x, z) - \log q_{\phi}(z|x)\end{aligned}$$

- Where $p(\epsilon)$ and $g()$ chosen such that $z \sim q_{\phi}(z|x)$
- Which has a simple Monte Carlo gradient:

$$\nabla_{\theta, \phi} \tilde{\mathcal{L}}_{\theta, \phi}(x; \epsilon)$$

Reparameterization

- With reparameterization, the ELBO can be written

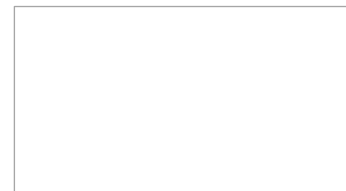
$$\begin{aligned}\mathcal{L}_{\theta,\phi}(x) &= \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)] \\ &= \mathbb{E}_{p(\epsilon)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)]\end{aligned}$$

- It turns out that the gradient of the Monte Carlo estimator of the reparametrized ELBO $\tilde{\mathcal{L}}_{\theta,\phi}(x)$ is an unbiased estimator of the true ELBO $\mathcal{L}_{\theta,\phi}(x)$ as follows

$$\begin{aligned}\nabla_{\phi} \mathcal{L}_{\theta,\phi}(x) &= \nabla_{\phi} \mathbb{E}_{p(\epsilon)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)] \\ &= \mathbb{E}_{p(\epsilon)} [\nabla_{\phi} \tilde{\mathcal{L}}_{\theta,\phi}(x)] \\ &\approx \nabla_{\phi} \tilde{\mathcal{L}}_{\theta,\phi}(x)\end{aligned}$$

- Unbiased estimator of the exact single-datapoint ELBO gradient:

$$\begin{aligned}\mathbb{E}_{p(\epsilon)}[\nabla_{\theta,\phi}\tilde{\mathcal{L}}_{\theta,\phi}(x;\epsilon)] &= \mathbb{E}_{p(\epsilon)}[\nabla_{\theta,\phi}(\log p_{\theta}(x,z) - \log q_{\phi}(z|x))] \\ &= \nabla_{\theta\phi}(\mathbb{E}_{p(\epsilon)}[\log p_{\theta}(x,z) - \log q_{\phi}(z|x)]) \\ &= \nabla_{\theta\phi}\mathcal{L}_{\theta,\phi}(x)\end{aligned}$$



Reparameterization and Backpropagation

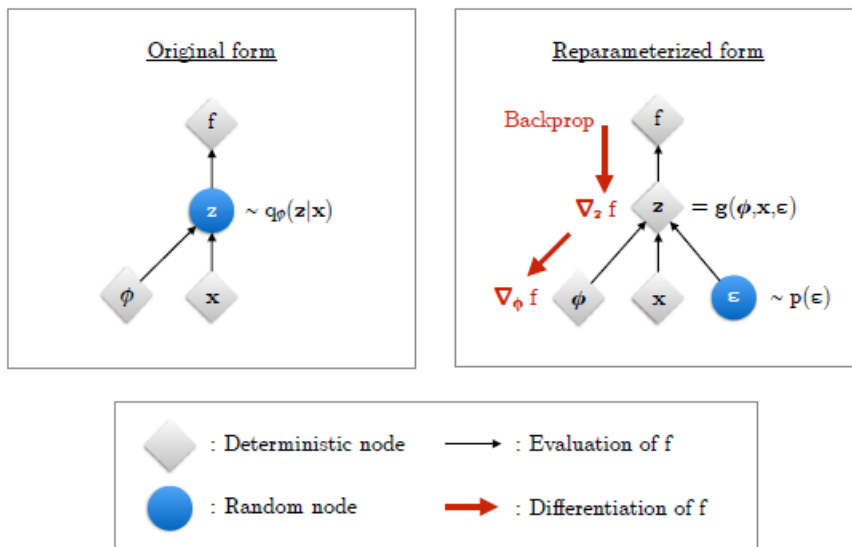
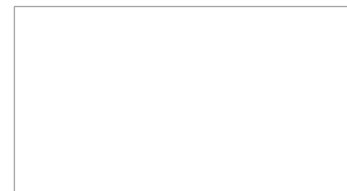


Illustration of the reparameterization trick. The variational parameters ϕ affect the objective f through the random variable $z \sim q_\phi(z|x)$. We wish to compute gradients $\nabla_\phi f$ to optimize the objective with SGD. In the original form (left), we cannot differentiate f w.r.t. ϕ , because we cannot directly backpropagate gradients through the random variable z . We can ‘externalize’ the randomness in z by re-parameterizing the variable as a deterministic and differentiable function of ϕ , x , and a newly introduced random variable ϵ . This allows us to ‘backprop through z ’, and compute gradients $\nabla_\phi f$.



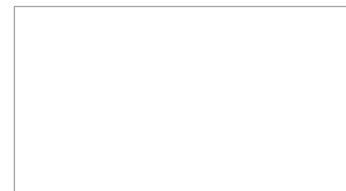
Reparameterization

- Under Reparameterization, density is given by :

$$\log q_{\phi}(\mathbf{z}|\mathbf{x}) = \log p(\boldsymbol{\epsilon}) - \log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} \right) \right|$$

- Important : choose transformations $g()$ for which the log-determinant transformation from $\boldsymbol{\epsilon}$ to $\mathbf{z}$ is computationally affordable / simple
- This log-determinant, similar to $g()$, is a function of $\mathbf{x}$, $\boldsymbol{\epsilon}$ and ϕ . The Jacobian matrix contains all first derivatives of the transformation from $\boldsymbol{\epsilon}$ to $\mathbf{z}$.

$$\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} = \frac{\partial (z_1, \dots, z_k)}{\partial (\epsilon_1, \dots, \epsilon_k)} = \begin{pmatrix} \frac{\partial z_1}{\partial \epsilon_1} & \dots & \frac{\partial z_1}{\partial \epsilon_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_k}{\partial \epsilon_1} & \dots & \frac{\partial z_k}{\partial \epsilon_k} \end{pmatrix}$$



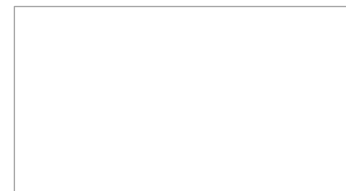
Factorized Gaussian Posterior

- A Common choice is a simple factorized Gaussian encoder :

$$\begin{aligned}(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) &= \text{EncoderNeuralNet}_{\phi}(\mathbf{x}) \\ q_{\phi}(\mathbf{z}|\mathbf{x}) &= \prod_i q_{\phi}(z_i|\mathbf{x}) = \prod_i \mathcal{N}(z_i|\mu_i\sigma_i^2)\end{aligned}$$

- After reparameterization, we can write :

$$\begin{aligned}\boldsymbol{\epsilon} &\sim \mathcal{N}(0, \mathbf{I}) \\ (\boldsymbol{\mu}, \log \boldsymbol{\sigma}) &= \text{EncoderNeuralNet}_{\phi}(\mathbf{x}) \\ \mathbf{z} &= \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}\end{aligned}$$



Factorized Gaussian Posterior

- The Jacobian of the transformation is :

$$\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} = \text{diag}(\boldsymbol{\sigma})$$

- Determinant of diagonal matrix is product of diag. entries.
- So the posterior density is :

$$\begin{aligned}\log q_{\phi}(\mathbf{z}|\mathbf{x}) &= \log p(\boldsymbol{\epsilon}) - \log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} \right) \right| \\ &= \sum_i \log \mathcal{N}(\epsilon_i; 0, 1) - \log \sigma_i\end{aligned}$$

Full-covariance Gaussian posterior

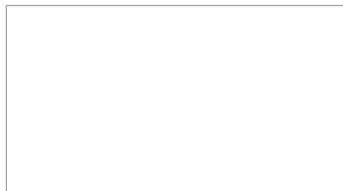
- The factorized Gaussian posterior can be extended to a Gaussian with full covariance :

$$q_{\phi}(\mathbf{z}|\mathbf{x}) = N(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

- A reparameterization of this distribution with a surprisingly simple determinant, is:

$$\begin{aligned}\boldsymbol{\epsilon} &\sim N(0, \mathbf{I}) \\ \mathbf{z} &= \boldsymbol{\mu} + \mathbf{L}\boldsymbol{\epsilon}\end{aligned}$$

- Where $\mathbf{L}$ is a lower (or upper) triangular matrix, with non-zero entries on the diagonal. This off-diagonal element define the correlations (covariance) of the elements in $\mathbf{z}$.



Full-covariance Gaussian posterior

- This reason for this parameterization of the full-covariance Gaussian, is that the Jacobian determinant is remarkably simple. The Jacobian is trivial :

$$\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} = \mathbf{L}$$

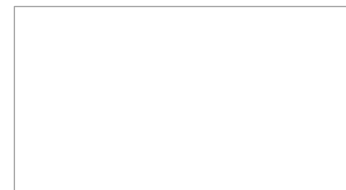
- And the determinant of a triangular matrix is simply the product of its diagonal terms. So:

$$\log |\det \left(\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} \right)| = \sum_i \log |L_{ii}|$$

Full-covariance Gaussian posterior

- This parameterization corresponds to the Cholesky decomposition of the covariance of $\mathbf{z}$:

$$\begin{aligned}\Sigma &= \mathbb{E}[\mathbf{z} - \mathbb{E}[\mathbf{z}]] (\mathbf{z} - \mathbb{E}[\mathbf{z}])^T] \\ &= \mathbb{E}[\mathbf{L}\boldsymbol{\epsilon}(\mathbf{L}\boldsymbol{\epsilon})^T] \\ &= \mathbb{E}[\mathbf{L}\boldsymbol{\epsilon}\boldsymbol{\epsilon}^T\mathbf{L}^T] \\ &= \mathbf{L}\mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^T]\mathbf{L}^T \\ &= \mathbf{L}\mathbf{L}^T\end{aligned}$$



Full-covariance Gaussian posterior

- One way to construct the matrix $\mathbf{L}$ is as follows :

$$\begin{aligned}(\boldsymbol{\mu}, \log \boldsymbol{\sigma}, \mathbf{L}') &\leftarrow \text{EncoderNeuralNet}_{\phi}(\mathbf{x}) \\ \mathbf{L} &\leftarrow \mathbf{L}_{mask} \odot \mathbf{L}' + \text{diag}(\boldsymbol{\sigma})\end{aligned}$$

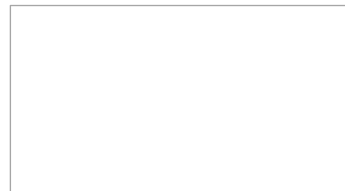
- $\mathbf{L}_{mask}$ is a masking matrix with zeros on and above the diagonal, and ones below the diagonal
- The log-determinant is identical to the factorized Gaussian case :

$$\log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} \right) \right| = \sum_i \log \sigma_i$$

Full-covariance Gaussian posterior

- Therefore, density equal to diagonal Gaussian case!

$$\begin{aligned}\log q_{\phi}(\mathbf{z}|\mathbf{x}) &= \log p(\boldsymbol{\epsilon}) - \log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} \right) \right| \\ &= \sum_i \log \mathcal{N}(\epsilon_i; 0, 1) - \log \sigma_i\end{aligned}$$



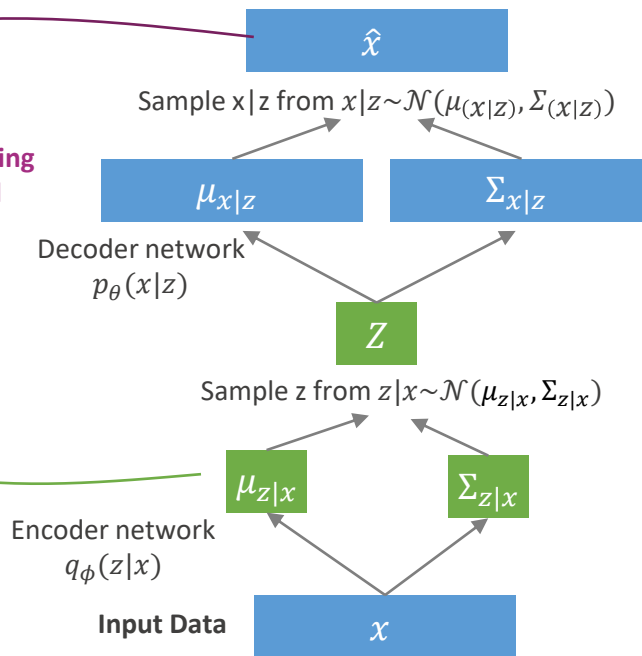
Putting it all together: maximizing the likelihood lower bound

Maximize likelihood of original input being reconstructed

$$\mathbb{E}_Z[\log p_\theta(x^{(i)}|z)] - D_{KL}(q_\phi(z|x^{(i)}) || p_\theta(z))$$

Make approximate posterior distribution close to prior

For every minibatch of input data: compute this forward pass, and then backprop!



Algorithm 1: Stochastic optimization of the ELBO. Since noise originates from both minibatch sampling and sampling of $p(\epsilon)$, this is a doubly stochastic optimization procedure. We also refer to this procedure as the Auto-Encoding Variational Bayes (AEVB) algorithm

Data:

$\mathcal{D}$: Dataset

$q_\phi(z|x)$: Inference model / Encoder Network

$p_\theta(x, z)$: Generative model / Decoder Network

Result:

θ, ϕ : Learned parameters

$(\theta, \phi) \leftarrow$ Initialize parameters

While SGD not converged do

$\mathcal{M} \sim \mathcal{D}$ (Random minibatch of data)

$\epsilon \sim p(\epsilon)$ (Random noise for every datapoint in $\mathcal{M}$)

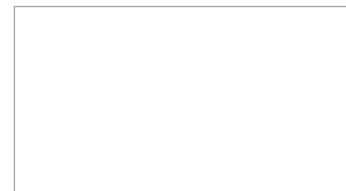
 Compute the reparameterized ELBO $\tilde{\mathcal{L}}_{\theta, \phi}(\mathcal{M}, \epsilon)$ and its gradients $\nabla_{\theta, \phi} \tilde{\mathcal{L}}_{\theta, \phi}(\mathcal{M}, \epsilon)$

 Update θ and ϕ using SGD optimizer

end

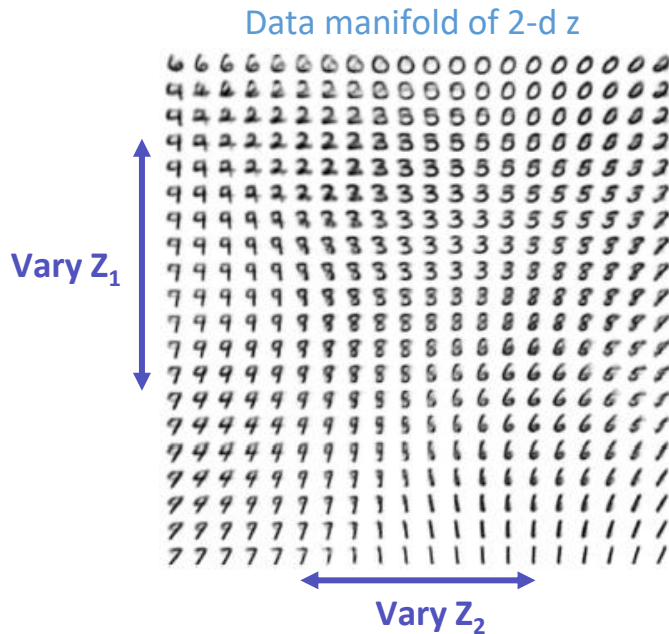
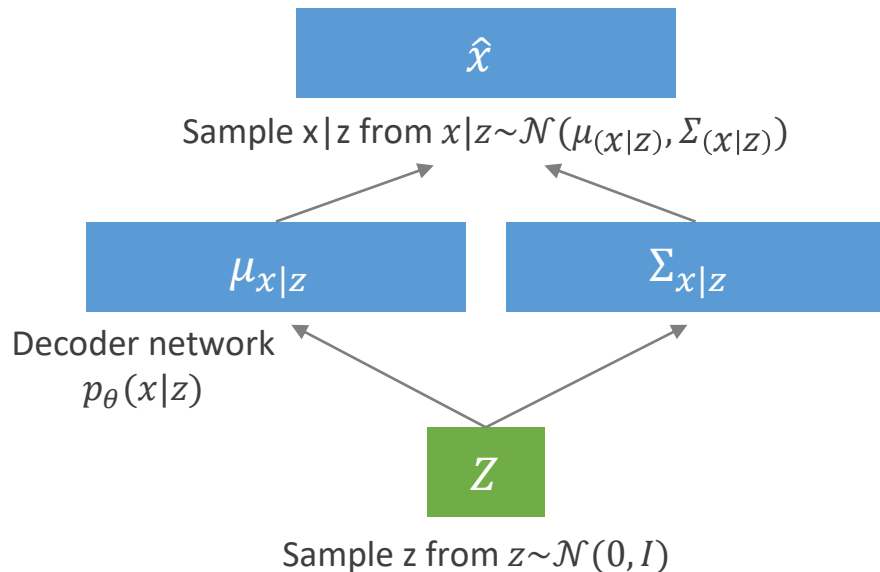
Variational Autoencoders

- Mathematical Preliminaries
- Autoencoders and Issues
- Derivations
- Reparameterization
- **Examples and Summary**



Variational Autoencoders: Generating Data!

Use decoder network. Now sample z from prior!



Reference: Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational Autoencoders: Generating Data!

Diagonal prior on $\mathbf{z}$ => independent latent variables

Different dimensions of $\mathbf{z}$ encode interpretable factors of variation

Also good feature representation that can be computed using $q_\phi(\mathbf{z}|\mathbf{x})$!

Degree of smile

Vary $\mathbf{Z}_1$



Vary $\mathbf{Z}_2$

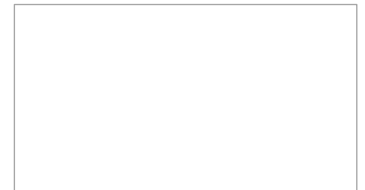
Head pose

Image Synthesis



VAEs can be used for image resynthesis, In this example by White, 2016, an original image (left) is modified in a latent space in the direction of a smile vector, producing a range of versions of the original, from smiling to sadness

White, T. (2016). "Sampling Generative Networks: Notes on a Few Effective Techniques". arXiv preprint arXiv:1609.04468



Sourjya Sarkar

“I want to talk to you.”

“i want to be with you.”

“i do nit want to be with you.”

i do n’t want to be with you.

she didn’t want to be with him.

he was silent for a long moment.

he was silent for a moment.

it was quiet for a moment.

it was dark and cold.

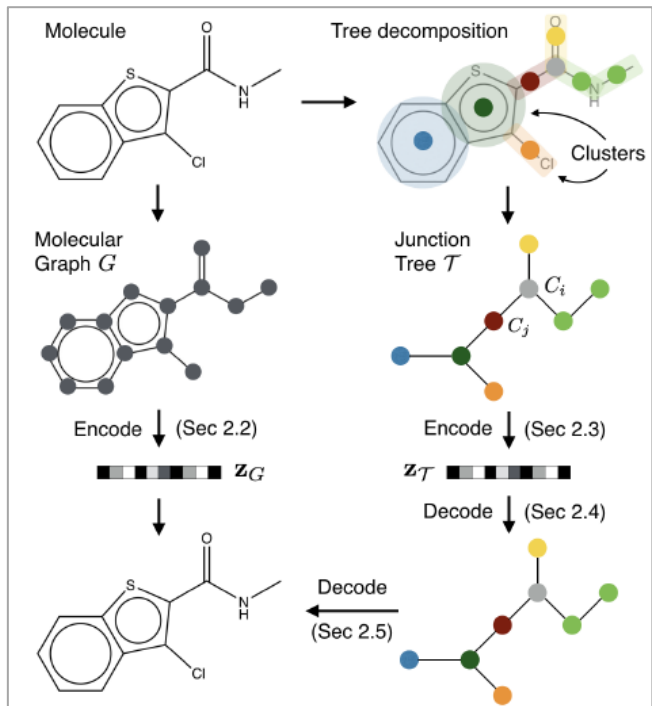
there was a pause.

it was my turn.

VAE was used to generating natural-language sentences from a continuous space by Bowman *et al.*, 2015. After training on text, the model was shown to successfully interpolate between sentences, and for imputation of missing words.

An Application of VAEs to interpolation between pairs of sentences, from (Bowman et al., 2015). The intermediate sentences are grammatically correct and the topic and syntactic structure are typically locally consistent.

Molecular Graph Generation



Overview of our molecule generation paradigm: A molecular graph G is decomposed into its junction tree $\mathcal{T}_G$, where each colored node in the tree represents a substructure in the molecule. We then encode both the tree and graph into their latent embeddings $z_{\mathcal{T}}$ and z_G . To decode the molecule, we first reconstruct junction tree from $z_{\mathcal{T}}$, and then assemble nodes in the tree back to original molecule, guided by z_G .

Pros and Cons

Probabilistic spin to traditional autoencoders => allows generating data.

Define as intractable density => derive and optimize a (variational) lower bound

Pros

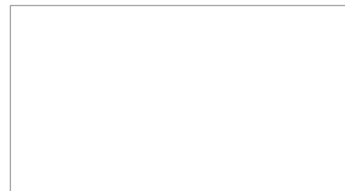
- Principled approach to generative models
- Allows inference of $q(z|x)$, can be useful feature representation for other task

Cons

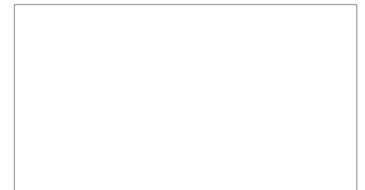
- Maximizes lower bound of likelihood: okay, but not as good evaluation as PixelRNN/PixelCNN
- Samples blurrier and lower quality compared to state-of-the-art (GANs)

Active areas of research:

- More flexible approximations, e.g., richer approximate posterior instead of diagonal Gaussian, e.g., Gaussian Mixture Models (GMMs)
- Incorporating structure in latent variables, e.g., Categorical Distributions



- D. P. Kingma and M. Welling, “*An Introduction to Variational Autoencoders*”, Foundations and Trends in Machine Learning, 2019.
- F.F. Li, J. Johnson and S. Yeung, “*Lecture11-Generative Models*”, 2019
http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture11.pdf





Trigonometry:

- $\sin \theta = \frac{\text{OPP}}{\text{HYP}}$, $\cos \theta = \frac{\text{ADJ}}{\text{HYP}}$, $\tan \theta = \frac{\text{OPP}}{\text{ADJ}}$
- $\csc \theta = \frac{1}{\sin \theta}$, $\sec \theta = \frac{1}{\cos \theta}$, $\cot \theta = \frac{1}{\tan \theta}$
- $1 = \sin^2 \theta + \cos^2 \theta$, $1 = \sec^2 \theta - \tan^2 \theta$, $1 = \csc^2 \theta - \cot^2 \theta$
- $\sin \frac{\theta}{2} = \pm \sqrt{\frac{1 - \cos \theta}{2}}$, $\cos \frac{\theta}{2} = \pm \sqrt{\frac{1 + \cos \theta}{2}}$, $\tan \frac{\theta}{2} = \pm \sqrt{\frac{1 - \cos \theta}{1 + \cos \theta}}$

Calculus:

- $\oint \vec{E} \cdot d\vec{l} = -\frac{d\phi_B}{dt}$
- $\oint \vec{B} \cdot d\vec{l} = \mu_0 I + \mu_0 \epsilon_0 \frac{d\phi_E}{dt}$
- $\oint \vec{E} \cdot d\vec{l} = (E + dE)h_f - E h_f = h_f dE$
- $\phi_B = BA = Bh_f dx$
- $\frac{d\phi_B}{dt} = h_f dx \frac{dB}{dt}$
- $\oint \vec{B} \cdot d\vec{l} = -(B + dB)h_2 + Bh_2 = -h_2 dB$
- $\phi_E = EA = E h_2 dx$, $\frac{d\phi_E}{dt} = h_2 dx \frac{dE}{dt}$
- $\oint \vec{E} \cdot d\vec{l} = -\frac{d\phi_E}{dt}$
- $h_f dE = -h_2 dx \frac{dB}{dt}$
- $\frac{dE}{dx} = -\frac{dB}{dt}$
- $\frac{\partial^2 E}{\partial x^2} = \mu_0 \epsilon_0 \frac{\partial^2 E}{\partial t^2}$

Electronics:

- Circuit diagram with NPN transistor, V_{BE} , V_{CB} , I_{BE} , I_B , V_{CE} , I_C
- $I_E = I_B + I_C$ and $\frac{I_C}{I_E} = \alpha$
- Thus: $I_B = I_E - I_C$, $I_B = I_E - \alpha I_E$, $I_B = I_E(1 - \alpha)$
- $\beta = \frac{I_C}{I_B} = \frac{I_C}{I_E(1 - \alpha)}$
- AC source, OPEN CIRCUIT, V_o , R_c , X_m

Optics:

- Ray diagram showing focal length f , object distance s , image distance s'
- $M = \frac{f}{s - f} = \frac{s' - f}{f}$
- $\frac{1}{f} = \frac{1}{s} + \frac{1}{s'}$, $m = \frac{i}{o} = \frac{s'}{s}$, $f = \frac{R}{2}$

Chemistry:

- $6CO_2 + 12H_2O \xrightarrow{\text{light, Chlorophyll}} C_6H_{12}O_6 + 6O_2$
- ATP cycle: ADP $\rightarrow$ 36P $\rightarrow$ RuBP $\rightarrow$ Glycolate $\rightarrow$ Serine $\rightarrow$ Glycine $\rightarrow$ 2PGlycolate + 3PGlycolate
- Photorespiration: RuBisCo
- $n \text{ HOOC-CH}_2\text{-CH}_2\text{-CH}_2\text{-COOH} + n \text{ H}_2\text{N-CH}_2\text{-CH}_2\text{-CH}_2\text{-NH}_2 \rightarrow \text{I-6-DIHEXANOIC ACID} + \text{I-6-DIAMINOHEXANE}$
- $n \text{ NH}_2\text{-CH}_2\text{-CH}_2\text{-COOH} \rightarrow \text{NYLON 6-6}$
- $2AlBr_3 + 3Cl_2 \rightarrow 2AlCl_3$
- $H-C-C-C(=O)-OH + H-C-C-OH \rightarrow H-C-C-C(=O)-O-C-C-H$
- PROPANOIC ACID, ETHANOL, ETHYL PROPANOATE

Mathematics:

- $\rho = \frac{m}{V}$
- $\frac{\partial^2 y}{\partial x^2} = \frac{1}{v^2} \frac{\partial^2 y}{\partial t^2}$, $v = \frac{\omega}{k}$, $v = f\lambda$, $\omega = 2\pi f$, $k = \frac{2\pi}{\lambda}$
- $y = A \sin(\omega t - kx)$, $B = B_{\text{max}} \sin(\omega t - kx)$
- $\frac{\partial^2 y}{\partial x^2} = -\frac{\partial^2 B}{\partial x^2} = -\frac{\partial^2 B_{\text{max}} \cos(\omega t - kx)}{\partial x^2}$
- $E = E_{\text{max}} \sin(\omega t - kx)$, $\frac{\partial E}{\partial x} = -k E_{\text{max}} \cos(\omega t - kx)$

Set Theory:

- Venn diagrams for $A \cap B$, $A \cup B$, $A - B$, $A' \cap B$
- $n(A \cup B) = n(A) + n(B) - n(A \cap B)$
- $n(A \cup B \cup C) = n(A) + n(B) + n(C) - n(A \cap B) - n(A \cap C) - n(B \cap C) + n(A \cap B \cap C)$

Thank you Q&A